

JetStream: Generating Query Accelerators for Existing Database Systems

Akhilesh Balasingam
MIT CSAIL
Cambridge, MA, USA
akhilvb@mit.edu

Amadou Ngom
MIT CSAIL
Cambridge, MA, USA
ngom@mit.edu

Geoffrey X. Yu
MIT CSAIL
Cambridge, MA, USA
geoffxy@mit.edu

Tim Kraska
MIT CSAIL
Cambridge, MA, USA
kraska@mit.edu

Abstract

Recent work has shown that LLMs can synthesize highly specialized database systems for fixed workloads, but existing approaches typically assume static data and replace the database’s native storage with generated representations. We present JetStream, a system for generating query-specific accelerators that instead extend an existing DBMS.

JetStream consists of three parts. First, a staged, measurement-driven agentic workflow generates and optimizes query-specific accelerators, including persistent auxiliary state when beneficial. Second, a fixed, engine-neutral substrate provides the common interfaces for execution, transaction coordination, state management, and maintenance. Third, a separate synthesis workflow generates engine-specific backend adapters that connect the substrate to the underlying engine. For stateful accelerators, JetStream also generates maintenance logic and uses a runtime policy to choose among incremental maintenance, rebuilds, and lazy repair as the database changes.

On TPC-H at SF=20, stateful accelerators generated by JetStream achieve an 833× geomean read-only speedup over DuckDB, compared with 34.07× for GenDB and 12.35× for Bespoke OLAP. Under TPC-H refreshes every 60 seconds, JetStream maintains a 375× geomean workload speedup. We also show that JetStream generalizes to new, unseen workloads, achieving geomean read-only speedups of 102× over DuckDB and 486× over PostgreSQL. These results show that aggressive generated specialization can be integrated seamlessly with existing DBMSes, and support dynamic workloads.

1 Introduction

Relational database systems are designed to be general-purpose: they support diverse schemas and data and can execute any query expressible in their SQL dialect. But this generality comes at the cost of performance. For example, while an aggregation can be generally implemented using a hash table, if the set of groups are known ahead of time, we could instead use a direct-mapped array and eliminate the hash-probing overhead altogether.

This example shows how knowledge of a query workload and its data can enable specialized execution strategies, which we call *accelerators*, that outperform general-purpose database systems. Historically, building such accelerators required substantial engineering effort, and by design, sacrificed generality. Such systems were thus typically built only for query workloads important enough to justify both the engineering effort and loss of generality (e.g., performance- or revenue-critical applications [1, 2, 36]). But recent advances in large language model (LLM) code generation [4, 24, 34] are changing this trade-off by making it practical to automatically construct specialized query accelerators for known workloads [17, 31, 35].

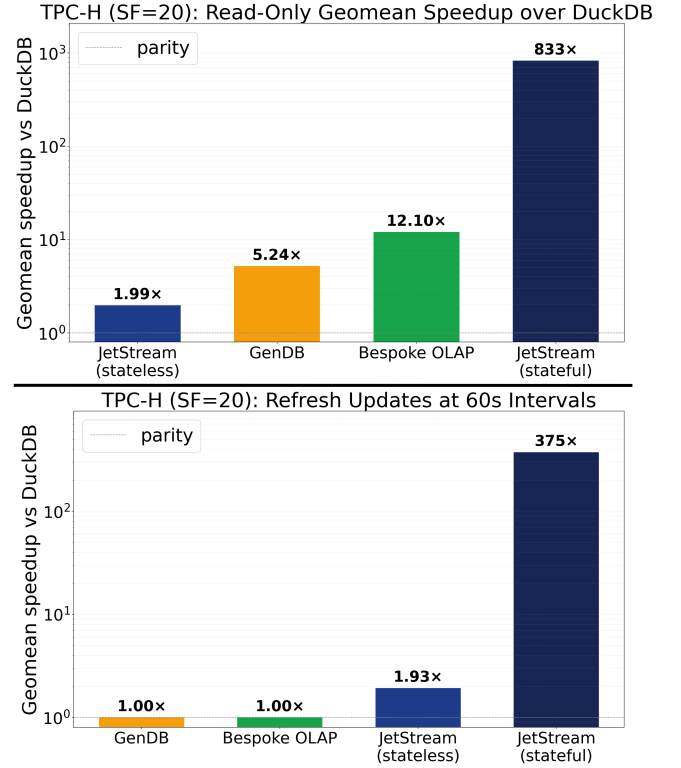


Figure 1: JetStream combines aggressive read acceleration with support for realistic write workloads. On TPC-H SF=20, stateful JetStream achieves substantially higher read-only speedup than prior generated systems (top). Under TPC-H refreshes every 60 seconds, JetStream retains a 375× geomean workload speedup over DuckDB, while systems without write maintenance fall back to parity with the base DBMS (bottom).

This raises a natural research question: can we obtain the performance benefits of specialization without giving up the generality and practicality of a conventional database system?

Systems such as GenDB [17] and Bespoke OLAP [35] take a step in this direction, both synthesizing query engines for a known workload by specializing its storage layout, data structures, and query implementations. However, as outlined in Table 1, both systems make design decisions that limit their practicality. First, they rewrite the base data itself, which complicates writes, potentially requires data to be rewritten if the workload changes, and forces users to migrate their data into the specialized format. Second, their

Table 1: No existing query accelerator synthesis system supports all desired practical capabilities.

System	Accelerates known queries	Supports ad-hoc queries	Preserves accel. under writes	Uses existing DBMS storage
Castor [11]	✓ Deductive program synthesis	✗	✗	✗
GenDB [17]	✓ Agentic synthesis workflow	✗	✗	✗
Bespoke OLAP [35]	✓ Agentic synthesis workflow	✓ Separate DBMS fallback	✗	✗
JetStream (this work)	✓ Agentic synthesis workflow	✓ Underlying DBMS execution	✓ Intelligent aux. state maintenance	✓ Using a general substrate

specialized data structures assume static data and lack a mechanism for incremental maintenance. After writes, they either return stale results or forgo their performance benefits altogether. Third, GenDB in particular sacrifices generality entirely by not supporting ad-hoc queries. We argue that a practical system leveraging synthesized query accelerators should retain the generality of a conventional DBMS, supporting writes and ad-hoc queries using its native storage, while delivering accelerated performance for known, pre-declared query templates.

Two challenges make achieving this difficult in practice. First, accelerator integration is susceptible to reward hacking [29]: modern coding agents can modify the system and invoke their own tests and benchmarks, making it unsafe to let the same agent optimize an accelerator and judge whether it is correct or faster. Apparent improvements can come from weakening correctness checks or exploiting measurement artifacts rather than actually improving the system, so correctness and performance evaluation must be owned by an external mechanism with fixed acceptance criteria. Second, integrating accelerators across multiple DBMSes requires more than generating query code, because engines expose different scan APIs, mutation paths, and transaction mechanisms. Encoding these details directly into every accelerator would tightly couple accelerator logic to a particular engine and force the synthesis process to repeatedly rediscover the same integration logic.

We present JetStream: a new practical system that synthesizes query-specific accelerators and incorporates them *within* an existing database system. JetStream addresses the first challenge with a measurement-driven synthesis workflow in which an external

orchestrator, rather than a coding agent, owns correctness and performance evaluation. It addresses the second with a fixed, engine-neutral substrate that separates query-specific accelerator logic from engine-specific database integration. This separation allows JetStream to specialize aggressively without replacing the underlying database. Accelerators can execute directly over native base tables or maintain query-specific auxiliary state, while the DBMS remains responsible for canonical storage, transactions, recovery, and concurrency. Queries without accelerators continue to execute normally, and accelerators can be added, removed, or regenerated without changing the representation of the base data. Like existing systems [17, 35], JetStream applies differential testing during accelerator synthesis and uses a final audit stage (Section 2.4) to empirically validate that its accelerators match the semantics of their corresponding SQL queries. JetStream builds on top of our prior work on Tailwind [38], which non-invasively integrates manually-written accelerators into existing database systems; JetStream is the first step towards automatically generating such accelerators.

As shown in Figure 1, our evaluation shows that this design provides both aggressive specialization and practical update support. On TPC-H at SF=20, stateful JetStream accelerators achieve an 833× geomean read-only speedup over DuckDB, compared with 34.07× for GenDB [17] and 12.35× for Bespoke OLAP [35]. With TPC-H refreshes every 60 seconds, JetStream retains a 375× geomean workload speedup. On SEC-EDGAR, a workload chosen to reduce potential LLM training-data overlap, JetStream shows that it can generalize well, achieving 102× and 486× geomean read-only speedups over DuckDB and PostgreSQL, respectively.

In summary, we make the following contributions:

- **A measurement-driven agentic workflow for database accelerator generation.** JetStream separates accelerator analysis, implementation, optimization, and validation; grounds optimization in profiling and controlled measurements; and safely synthesizes multiple query accelerators concurrently.
- **Generated accelerators that extend, rather than replace, an existing DBMS.** We introduce an engine-independent substrate together with generated backend adapters that allow specialized query code to execute directly over native database storage. The same framework supports stateless or stateful accelerators and single- or multi-threaded execution.
- **A write-aware framework for generated auxiliary state.** Stateful accelerators include query-specific maintenance logic that incrementally updates, rebuilds, or invalidates auxiliary state based on the expected cost of writes and future queries, together with a transactional protocol that keeps generated state consistent with committed base data after updates.

2 System Overview & Details

JetStream takes as input a query workload together with a synthesis configuration specifying the target database engine, whether execution is single- or multi-threaded, and whether persistent auxiliary state is permitted. As shown in Figure 2, JetStream generates query-specific accelerators for the workload and the engine-specific backend needed to deploy them on an existing DBMS. Accelerators can execute directly over native DBMS storage or additionally maintain query-specific auxiliary structures.

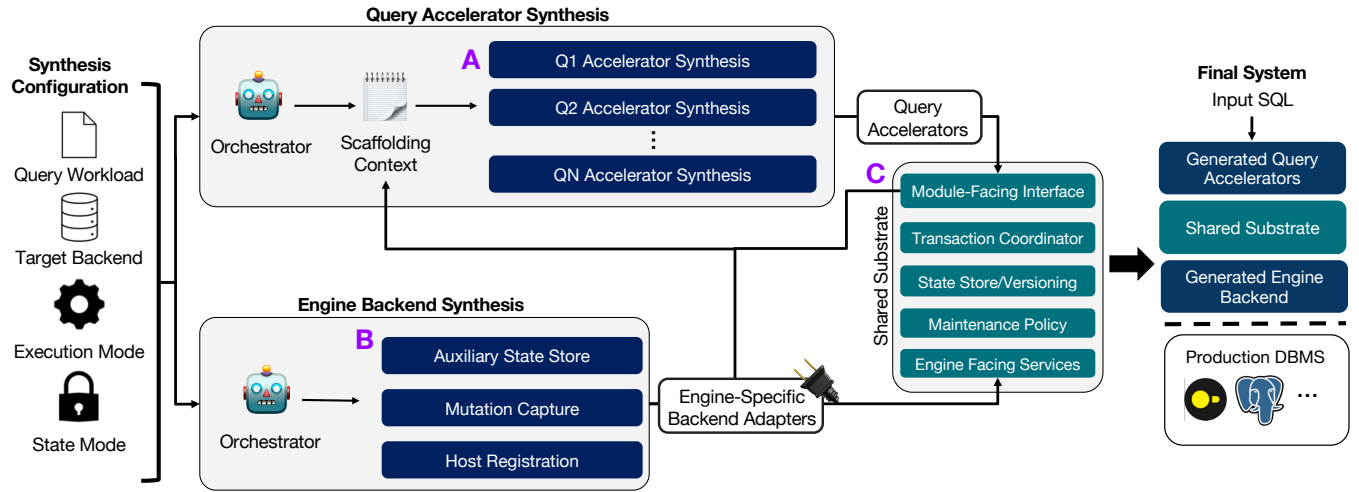


Figure 2: JetStream system overview. (A) Concurrent synthesis of query-specific accelerators (Section 2.1); (B) generation of engine-specific backend adapters (Section 2.2); and (C) the fixed shared substrate connecting generated accelerators to the underlying DBMS (Section 2.2).

The generated accelerators and backend compose through a fixed, engine-neutral substrate, while the underlying DBMS remains responsible for canonical storage and transaction processing. The remainder of this section follows the three components highlighted in Figure 2.

2.1 Agentic Workflow

JetStream generates each query accelerator through a staged, measurement-driven workflow. Although modern coding agents can inspect a repository, modify code, and invoke tests and benchmarks within a single session, accelerator synthesis is fundamentally a multi-stage design problem with delayed and noisy feedback. Early decisions about execution strategy and, when applicable, auxiliary state organization determine the space of implementations that can be explored, but their consequences only become observable after a concrete system is built and exercised under load. Conflating these phases into a single unconstrained agent loop risks premature commitment to suboptimal designs and makes it difficult to attributed observed performance to specific design choices. Moreover, if the agent is responsible for both modifying the system and evaluating its correctness and performance, the evaluation signal becomes part of the agent’s own decisions, weakening the reliability of acceptance criteria iterations. Empirically, we found that even strong coding agents, including Codex with GPT-5.6.-Sol, did not reliably synthesize a complete set of query accelerators from only a workload description and a target database engine.

JetStream instead structures accelerator generation as the workflow shown in Figure 3. The pipeline separates high-level design, implementation, optimization, and audit, while an external orchestrator owns correctness and performance evaluation.

2.1.1 Staged Accelerator Synthesis. Each stage in Figure 3 is executed by a separate instance with stage-specific context, instructions, and tools. The analysis agent receives the query, synthesis

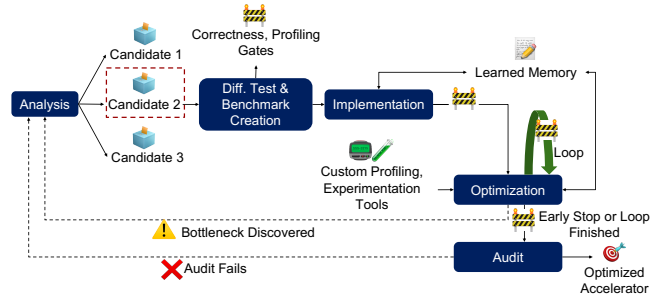


Figure 3: JetStream’s query-accelerator synthesis workflow.

configuration, and repository context needed to understand the available substrate and existing accelerator implementations. Later agents receive the artifacts produced by earlier stages, together with the tools relevant to their task, such as build, test, benchmark, and profiling interfaces. This keeps each agent focused on a narrow objective while preserving the information needed across stages.

The analysis stage proposes several materially different accelerator designs. These candidates specify the high-level execution strategy, and, for stateful accelerators, the organization of auxiliary state and its maintenance strategy. JetStream selects among these alternatives before committing to an implementation.

The implementation agent realizes the selected design against the shared substrate. Before optimization begins, JetStream also constructs the differential tests and benchmarks used to evaluate the accelerator, preventing later optimization from weakening correctness checks or changing the benchmark to favor a candidate. After optimization converges, a separate audit agent expands validation beyond the tests visible during implementation and optimization. An audit failure returns the pipeline to analysis; otherwise, the accelerator is finalized.

2.1.2 Evidence-Guided Optimization. Optimization proceeds through measured candidate attempts rather than free-form code refinement. For each attempt, the orchestrator builds the candidate, runs the required correctness gates, and compares it against the best validated implementation using controlled A/B benchmarks. A candidate becomes the new best only if it remains correct and improves the optimization objective.

When aggregate latency does not reveal the bottleneck, the optimization agent can synthesize additional instrumentation and experiments, such as phase timers and query-specific counters. These measurements help distinguish costs from base scans, auxiliary-state lookup, decoding, synchronization, or maintenance. If the evidence shows that the current state organization or execution strategy has reached a structural limit, the pipeline returns to analysis rather than continuing to accumulate local optimizations.

For stateful accelerators, the optimization signal includes both reads and writes. Rather than evaluating every attempt at a single write frequency, JetStream samples multiple write frequencies across attempts, exposing the optimizer to different read-versus-maintenance tradeoffs.

The pipeline also carries evidence forward across attempts. Failed hypotheses, performance regressions, and identified bottlenecks are retained as negative experience, while successful changes and useful profiles are preserved. This evidence is compressed between attempts so later agents avoid repeating failed work without carrying the full conversational history.

2.1.3 Concurrent Workload Synthesis. JetStream synthesizes multiple query accelerators concurrently rather than processing a workload serially. Each query runs as an asynchronous worker that advances independently through analysis, implementation, and optimization, so a slow query does not impose a stage barrier on the rest of the workload.

Naively running multiple coding agents against one repository is unsafe because query implementations ultimately compile into a shared library and touch common registration and build surfaces. JetStream therefore gives each worker a private source worktree, build directory, and agent runtime. Query-owned files remain private, shared registration files are coordinated through an explicit lease, and workers cannot modify another query’s generated implementation.

The orchestrator also distinguishes failures in a worker’s own code from failures caused by shared or peer state. In the latter case, it restores the affected files from the most recent validated snapshot rather than spending agent turns debugging foreign code. Workers additionally receive a compact shared repository context describing stable integration points and existing accelerators, reducing repeated repository discovery while keeping the individual synthesis trajectories isolated.

2.2 System Architecture

JetStream separates query-specific accelerator logic from engine-specific database integration through a fixed shared substrate. Generated accelerators do not call DBMS APIs directly; instead, they interact only with engine-neutral substrate interfaces, while generated backend adapters translate those interfaces to the native

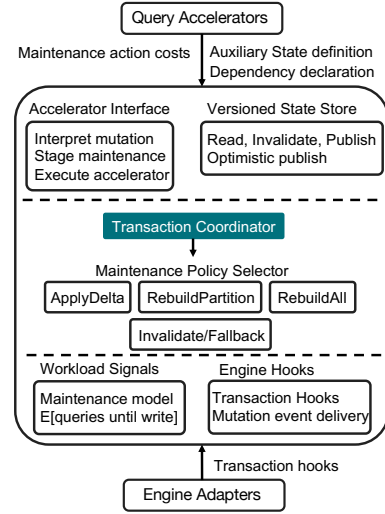


Figure 4: The fixed shared substrate separates query-specific accelerator logic from engine-specific database integration.

mechanisms of the target database. This yields a three-layer runtime structure: query accelerators define *what* computation and auxiliary state to use, the substrate provides common transaction coordination and state-management mechanisms, and the backend adapter implements the engine-specific I/O, storage, and transaction hooks.

At runtime, each accelerator is exposed through the host DBMS using its normal extension mechanism—for example, as a table function in DuckDB or an extension function in PostgreSQL. Invoking this entry point dispatches into the generated query module, which executes through the shared substrate and reaches the underlying engine only through the generated backend adapter.

2.2.1 Shared Accelerator Substrate. Figure 4 details the fixed substrate and the interfaces supplied by generated accelerators and engine backends. Its transaction coordinator registers accelerators and routes mutations according to their declared dependencies; its state layer tracks the validity and versions of its auxiliary state; and its maintenance layer selects how state should be repaired after writes. The substrate additionally exposes portable native-scan requests that the generated backend translates into engine-native table scans.

The same substrate supports both stateless and stateful accelerators. Stateless accelerators request the columns and predicates they need through an engine-neutral scan interface; the backend maps that request onto the DBMS’s native storage scan and streams the resulting data to the generated accelerator. Stateful accelerators additionally store persistent query-specific structures through the substrate’s state interface. These structures are organized into *partitions*, where a partition is a query-defined unit of auxiliary state, such as a month, date range, or key range. This partitioning is independent of any physical partitioning used by the underlying DBMS and allows maintenance to target only the affected portion of an accelerator’s state. This boundary keeps query-specific state

and execution logic independent of engine-specific storage and transaction mechanisms.

2.2.2 Generated Engine Backends. For each target DBMS, JetStream generates a backend that implements the engine-facing side of the substrate. The backend provides four core pieces of functionality: native base-table scans, capture and translation of inserts, updates, and deletes, transaction lifecycle callbacks, and storage for auxiliary-state images. It also provides the extension or table-function glue needed to expose generated accelerators through the host database.

The native scan path stays inside the DBMS. In DuckDB, the backend binds scan requests to the engine’s native table-storage APIs, including available filter pushdown, and streams data directly to the generated accelerator. PostgreSQL uses its table access methods to perform the corresponding scans before passing the results through the same engine-neutral representation. Thus, state construction operate over the canonical base tables without reissuing the query through a client SQL interface.

The auxiliary-state representation adapts to the execution model of the underlying engine. DuckDB runs in process with the accelerator runtime, so the backend can retain typed in-memory state without serialization on the query hot path. PostgreSQL uses separate backend processes and shared memory, so the backend serializes auxiliary state at the process boundary and reconstructs the same logical query state before the accelerator consumes it.

Finally, the backend translates native write and transaction events into the common substrate protocol. Inserts, updates, and deletes become engine-neutral mutations, while pre-commit, commit, abort, and savepoint events drive the corresponding maintenance lifecycle. The adapter is the only layer that knows how these events are exposed by a particular engine. For DuckDB, which does not currently expose all transaction-update hooks required by our maintenance protocol, we add a small set of hooks to its transaction-handling internals to surface these events to the generated backend.

2.3 Auxiliary State Maintenance

Stateful accelerators must keep their auxiliary structures synchronized with the canonical base tables without making every write pay the cost of fully rebuilding state. JetStream therefore separates maintenance into two parts: a policy that chooses how an accelerator should respond to a transaction’s writes, and a transactional protocol that stages and publishes the resulting state changes. Figure 5 summarizes where these decisions occur in the read and write paths.

2.3.1 Maintenance Policy. When a transaction modifies the database, JetStream first filters the write using each accelerator’s declared dependencies and query-specific interpretation of the mutation. Writes that cannot affect an accelerator require no maintenance. Relevant writes are accumulated over the transaction together with the number of rows touched and the set of auxiliary-state partitions affected.

At pre-commit, each accelerator chooses one maintenance strategy for the transaction. It can incrementally update the affected state, rebuild only the affected partitions, rebuild the full auxiliary structure, or invalidate the affected state and defer to the read path (where a subsequent query can lazily publish a repaired state). The

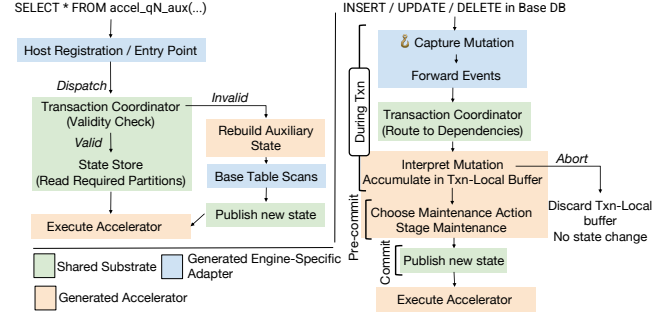


Figure 5: Read and write paths for a stateful accelerator.

available choices depend on the generated accelerator: for example, incremental maintenance is used only when the accelerator provides a valid update path, while partition rebuilding requires that the affected state can be localized.

To choose among the valid actions, the substrate maintains a runtime cost model for each accelerator. The model combines measurements of query and maintenance costs with properties of the current transaction and workload behavior. Thus, the decision can change as writes become larger or more frequent rather than relying on a fixed maintenance policy. For an action a ,

$$C(a) = C_{\text{immediate}}(a) + C_{\text{future}}(a),$$

where the first term captures work paid by the current write and the second captures the expected read-side cost before the next write.

In our implementation, these estimates use simple measured models:

$$C_{\text{delta}} = c_0 + c_r R, \quad C_{\text{part}} = P c_p, \quad C_{\text{full}} = c_f,$$

while invalidation trades low immediate cost against future read overhead:

$$C_{\text{invalidate}} = c_i + Q (T_{\text{fallback}} - T_{\text{hit}}).$$

Here, both R , the number of rows touched, and P , the number of affected auxiliary-state partitions, come from the current transaction. The maintenance-cost parameters and query latencies are calibrated from observed executions, and Q is estimated from the observed number of queries between writes. The lowest-cost eligible action is selected. This simple model captures the basics of the central tradeoff: incremental maintenance is attractive for small writes, localized rebuilds for small affected regions, while invalidation is cheap immediately but gives up future accelerated reads.

2.3.2 Transactional Maintenance Protocol. The transactional protocol determines *when* changes to the auxiliary state become visible. When the DBMS observes an insert, update, or delete, the generated backend converts the engine-specific event into a common mutation representation containing the affected table, operation type, old and new row values, and changed columns. The shared substrate routes the mutation only to accelerators that depend on the modified table, as shown in Figure 5.

Each accelerator then interprets the mutation according to its own state organization, identifies the affected partitions, and accumulates query-specific changes in transaction-local state. These staged changes remain invisible to other transactions. At pre-commit, the accelerator applies the maintenance decision described above to the accumulated transaction-local changes. For incremental maintenance, it materializes replacement state for the affected partitions; for a rebuild, it records it to lazily rebuild on the next missed read.

At commit, JetStream installs the resulting state. Rebuilds rescan the canonical base tables, reconstruct the required partitions or full structure, invalidate the corresponding old state, and publish the replacement. Incremental updates are published optimistically: each new partition image is associated with the version from which it was derived and is installed only if that version is still current.

If another writer has already advanced the same partition, publication fails rather than overwriting newer state. JetStream invalidates the conflicting partition and allows the base-table transaction to commit normally. Subsequent queries that require that partition execute through the base-table fallback path until a later write successfully updates or rebuilds the state. If the transaction aborts, all accumulated mutations and staged auxiliary state are discarded.

2.4 Correctness Assumptions

JetStream requires every generated accelerator to be semantically equivalent to its reference SQL query. This requirement applies to both stateless accelerators and stateful accelerators that answer queries using auxiliary structures. During synthesis, accelerator outputs are compared against the underlying DBMS using differential tests, and the final audit expands this test suite with cases that were not visible during optimization. Candidates that fail these checks are rejected.

For stateful accelerators, correctness also requires that auxiliary-state visibility respect the transaction semantics of the underlying DBMS. Updates produced by an uncommitted transaction remain transaction-local and are never visible to other readers. If the transaction aborts, its buffered mutations and any staged auxiliary-state changes are discarded. Whenever the system cannot establish that an auxiliary structure is safe to use, it falls back to execution over those tables.

3 Evaluation

Our evaluation answers four questions:

- How much does JetStream accelerate analytical queries? (Section 3.2)
- How does its performance change as writes become more frequent? (Section 3.3)
- Does the maintenance policy adapt effectively to different write patterns? (Section 3.3)
- What kinds of optimizations does the agentic pipeline discover? (Section 3.4)

3.1 Experimental Setup

We evaluate JetStream on two query workloads: TPC-H at SF=20 [30] and SEC-EDGAR at SF=3. TPC-H provides a standard analytical workload with well-understood query patterns and optimization opportunities. We additionally use SEC-EDGAR, following GenDB

[17], to evaluate whether the synthesis pipeline generalizes beyond workloads that are likely to be represented in LLM training data. SEC-EDGAR is constructed from real-world SEC financial-statement data from 2022-2024. GenDB [17] generates 1000 SQL-Smith queries over this dataset, which we use for JetStream evaluation.

Experiments run on a dual-socket server with two Intel Xeon Gold 6230 CPUs (40 physical cores, 80 hardware threads total), 376 GiB of memory, and a 1 TB NVMe SSD. Unless otherwise noted, query experiments use a single thread.

To illustrate the generalizability of JetStream, we evaluate on two DBMS backends: DuckDB and PostgreSQL 18.4. We compare against these base systems, as well as against GenDB [17] and BespokeOLAP [35], and evaluate both stateless and stateful variants of JetStream.

The synthesis pipeline uses GPT-5.6 Terra Pro for the analysis stage and DeepSeek V4 Pro for implementation, optimization, debugging, testing, and audit. Analysis runs through the OpenAI Agents SDK, while the code-oriented stages use the Codex exec harness.

Synthesis cost. Synthesizing the DuckDB and PostgreSQL backends cost \$4.15 and \$3.15, respectively. Generating all 22 TPC-H stateful accelerators cost \$41.63, while the selected SEC-EDGAR stateful accelerators cost \$10.92. The 22 stateless TPC-H accelerators cost \$15.22.

3.2 Steady-State Read Performance

We first evaluate the steady-state read performance of generated accelerators on TPC-H and SEC-EDGAR. Figure 6 reports per-query speedup on all 22 TPC-H queries at SF=20. Stateless JetStream accelerators, which execute directly over the host DBMS without persistent auxiliary state, achieve a 1.99× geomean speedup over single-threaded DuckDB. Allowing the pipeline to synthesize query-specific auxiliary state increases the geomean speedup to 833×, compared with 34.07× for GenDB and 12.35× for Bespoke OLAP.

The largest gains come from eliminating repeated analytical work rather than only making that work faster. A conventional TPC-H execution may scan and join large portions of the SF=20 database before producing a small aggregate result. Stateful accelerators generated JetStream instead stores query-specific, parameter-independent sufficient statistics, such as per-date aggregates, compact multidimensional summaries, and range trees. On a hit, the accelerator would only have to read a few state partitions and performs a short final computation. For many queries, this reduces a multi-second scan, join, and aggregation pipeline to a sub-millisecond or few-millisecond lookup-and-compose path.

This more aggressive specialization also explains much of the gap with GenDB and Bespoke OLAP. Those systems generate workload-specific storage first, and then focus on speeding up query execution by specializing operator implementations. JetStream goes further by additionally synthesizing persistent query-specific state that removes entire scans, joins, and aggregates from the read path. Additionally, the state that JetStream generates is organized around the query attributes that vary across parameter bindings. For example, TPC-H Q14 is parameterized by ship date, so JetStream maintains per-month revenue statistics and answers a bound query

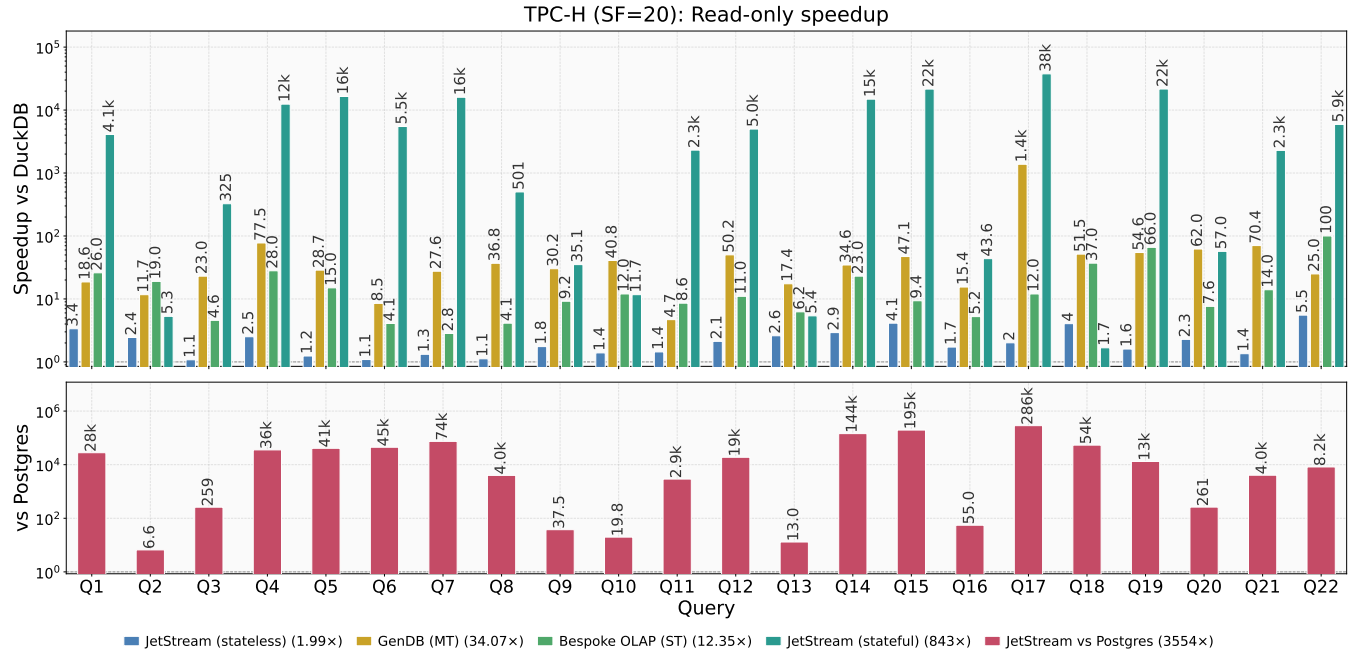


Figure 6: JetStream achieves large read-only speedups across TPC-H. Per-query speedup at SF=20 relative to DuckDB and PostgreSQL. Stateful JetStream achieves an 833× geomean speedup over DuckDB and 3544× speedup over PostgreSQL, substantially exceeding multi-threaded GenDB (34.07×) and Bespoke OLAP (12.35×), while stateless JetStream achieves 1.99×.

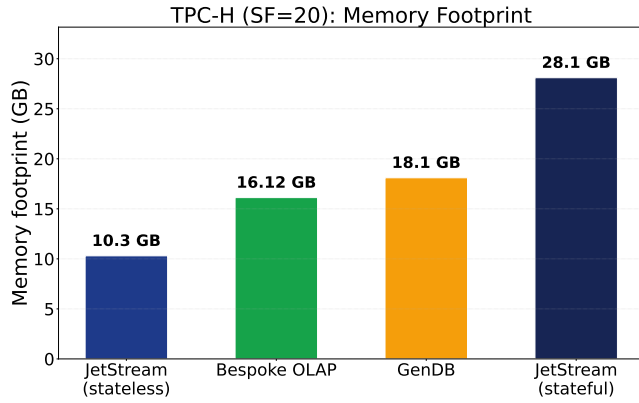


Figure 7: Memory footprint on TPC-H at SF=20. Stateful JetStream uses additional memory to maintain query-specific auxiliary state.

by reading the corresponding month and computing the final ratio. Range-oriented queries, for example, use trees or multidimensional summaries keyed by the attributes that define those ranges. The pipeline searches over these state organizations directly, allowing different queries to use different structures rather than conforming to a fixed, predetermined physical design, as GenDB and Bespoke OLAP do.

Naturally, these gains come with a space cost. As shown in Figure 7, stateful JetStream uses more memory than the stateless configuration and the generated-system baselines. This reflects

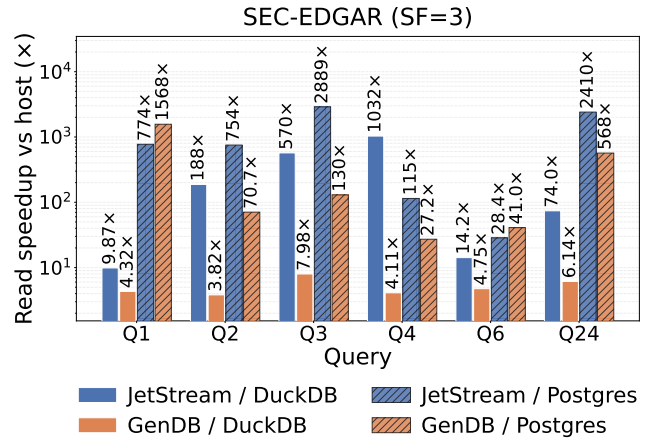


Figure 8: JetStream performance on unseen SEC-EDGAR queries. JetStream achieves a 102× geomean read-only speedup over DuckDB and 486× over PostgreSQL.

an explicit time-space tradeoff: JetStream uses additional query-specific auxiliary state to move more computation off the read path. The speedups in Figure 6 measure warm reads with valid auxiliary state and exclude the one-time cost of constructing that state. We account for the cost of keeping the state current under writes in Section 3.3.

Figure 8 shows that the gains are not specific to TPC-H. On SEC-EDGAR, JetStream outperforms GenDB on most evaluated queries

against both DuckDB and PostgreSQL. Since SEC-EDGAR was chosen to reduce the chance that the workload itself is well represented in LLM training data, the result therefore provides evidence that the pipeline can adapt to unfamiliar schemas, storage layouts, and query structures rather than relying only on well-known TPC-H optimizations. For example, SEC-EDGAR Q1 leads the pipeline to exploit the observed dictionary-over-flat representation of string columns; we discuss this case in Section 3.4. The gains also persist under updates: with point writes every 0.5 s, JetStream achieves an 8.8× geomean workload speedup over DuckDB and 4.0× over PostgreSQL, including both query and maintenance time.

3.3 Performance Under Updates

We next evaluate how well JetStream preserves its read advantage as the underlying database changes. We consider two complementary update regimes. First, we use the standard TPC-H refresh functions RF1 and RF2, which insert and delete batches corresponding to approximately 0.1% of the database. These are comparatively large updates and stress the cost of rebuilding or incrementally maintaining substantial portions of auxiliary state. Second, we use synthetic point writes to isolate the behavior of fine-grained maintenance under much smaller but more frequent updates.

Figure 9 shows the TPC-H refresh experiment. Even when a full RF update is issued once per minute, JetStream retains a 375× geomean workload speedup over DuckDB; at 120- and 300-second intervals, the geomeans rise to 629× and 733×. The decrease with shorter intervals reflects the larger fraction of time spent maintaining or repairing auxiliary state, but the read savings still dominate for most queries.

This result also highlights the difference from read-only generated systems. GenDB and Bespoke OLAP can specialize aggressively when their generated state remains static, but do not provide a general mechanism for keeping that state consistent under updates. In contrast, JetStream can incrementally update, rebuild, or invalidate and lazily repair state depending on the cost of the current write. As a result, the system can preserve much of its steady-state read benefit even when the base database is actively changing.

The point-write experiment in Figure 10 examines the opposite regime: small updates arriving much more frequently. For queries whose auxiliary state admits cheap incremental maintenance, JetStream continues to provide large speedups even at sub-second write intervals. Unlike the RF workload, these updates typically touch only a small number of rows and state partitions, allowing the maintenance policy to avoid expensive rebuilds.

Figure 11 illustrates why the two update regimes—fine-grained point writes and larger bulk writes (such as those in the TPC-H refresh workload)—favor different maintenance strategies. Phase A is read-heavy with periodic point writes, Phase B remains read-heavy but switches to periodic bulk writes, Phase C increases the frequency of point writes, and Phase D returns to the read-heavy point-write regime.

During the point-write phases, only a small amount of auxiliary state is affected, so the selector favors incremental maintenance and closely tracks the always-incremental policy. An always-full-rebuild policy performs poorly in this regime because each small

update incurs the cost of reconstructing much more state; when writes arrive frequently, rebuild work dominates the workload.

The tradeoff reverses under bulk writes. Applying many individual deltas becomes expensive, causing the always-incremental policy to lose much of its advantage. JetStream instead switches to lazy rebuilding, after which accelerated reads resume. When the workload returns to point writes, the selector switches back to incremental maintenance. Thus, neither fixed strategy performs well across both update regimes: JetStream adapts its maintenance action to the size and shape of the current write workload, as described in Section 2.3.

3.4 Generated Accelerator Examples

We next highlight three representative optimizations selected by JetStream. These examples span both kernel optimization and auxiliary-state synthesis, and illustrate how the pipeline combines workload structure, profiling evidence, and read/write tradeoffs to arrive at non-obvious implementations.

Before discussing the generated workloads, we briefly summarize the relevant data in the two workloads. SEC-EDGAR is derived from public company filings. The pre table contains filing-line records; adsh identifies a filing, stmt is the financial-statement category, rfile is the report file category, and line gives the line number within the filing data. TPC-H models a sales database. The customer table stores customers, orders stores orders placed by those customers, and lineitem contains the individual items within each order. part stores product attributes such as brand, size, and container type. The examples below show how JetStream reorganizes information from these base tables into query-specific state that reduces the work at read time.

SEC-EDGAR Q1: layout-aware distinct aggregation. SEC-EDGAR Q1 summarizes filing-line records by statement and report-file category. For each group, it computes the number of records, the number of distinct filings, and the average line number. Evaluating it normally would require scanning roughly 9.6M qualifying rows, grouping on two low-cardinality string attributes, and tracking distinct values of the high-cardinality filing identifier adsh. Profiling showed that 99.7% of input chunks used dictionary-encoded group keys with flat payload columns, while only about 379K distinct filing identifiers appeared across the 9.6M rows.

Based on this layout, JetStream maintains persistent query-specific aggregation state. It maps each adsh once into a dense integer ID, and, for each statement and report-file group, stores the running row count, line-number sum, and compact distinct-filing representation (a bitmap) over those dense IDs. Because the observed input is overwhelmingly dictionary-encoded on the grouping columns, the accelerator can operate largely on compact IDs instead of repeatedly hashing and comparing strings. At read time, the result is produced directly from the maintained group state, by just looking at the bitmap cardinality and computing the average line number from the running count.

The optimization emerged through several failed attempts. Earlier variants introduced a custom string hash set or fast paths for layouts that profiling later showed were uncommon; both failed to improve the dominant cost and in some cases regressed performance. Phase timers and layout counters instead pointed to

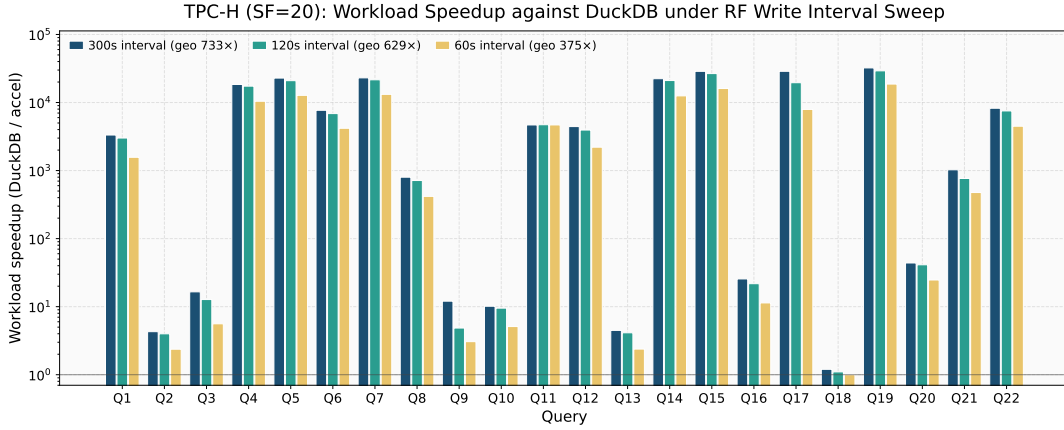


Figure 9: JetStream retains large speedups under TPC-H refreshes. Per-query workload speedup over DuckDB at SF=20 for RF1/RF2 refresh intervals of 300 s, 120 s, and 60 s. Geomean speedups: 733 $\times$, 629 $\times$, and 375 $\times$, respectively.

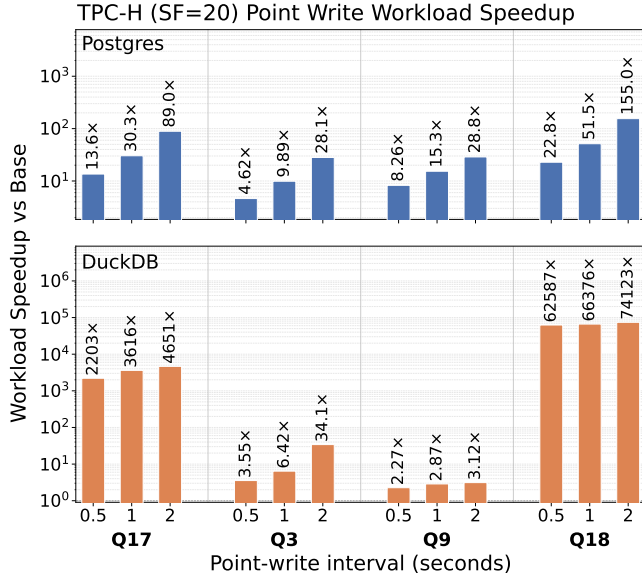


Figure 10: Per-query speedup over DuckDB and Postgres for JetStream under synthetic point-write workload.

repeated distinct-string processing on the mixed dictionary-over-flat path, which led to the retained map-and-bitmap design.

TPC-H Q13: q-gram postings with sparse histogram updates. TPC-H Q13 asks how many customers have each possible number of qualifying orders. For a bound comment pattern, it excludes orders whose comments match that pattern, left-joins the remaining orders to customers, counts the qualifying orders per customer, and then constructs a histogram over those counts. The expensive work is therefore both parameter-dependent and spread across several operators: comment filtering, the customer-order join, and two levels of aggregation.

JetStream separates the parameter-independent work from the parameter-dependent correction. Its persistent auxiliary state contains a q-gram inverted index over order comments, a mapping

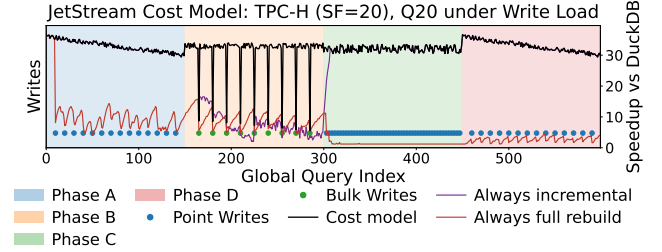


Figure 11: JetStream adapts maintenance strategy to workload. Phase A is read-heavy with periodic point-writes; Phase B is read-heavy with periodic bulk writes; Phase C is write-heavy with frequent point-writes; Phase D returns to a read-heavy workload with periodic point writes.

from orders to customers, each customer’s baseline order count, and a baseline histogram of customer order counts. At read time, the bound comment pattern is decomposed into q-grams, whose posting lists identify candidate matching orders. JetStream verifies the exact LIKE predicate only on those candidates, maps the matched orders to affected customers, adjusts only those customers’ counts, and applies sparse corrections to the baseline histogram. This avoids rescanning all orders and recomputing counts for every customer.

The analysis stage considered simpler alternatives, including a compact customer-comment intermediate and an inverted index followed by a full scan over customers. These designs removed part of the original work but left a scan-sized residual cost. The selected design combines selective search with sparse histogram correction, removing both the orders scan and the all-customer scan for selective bindings. It achieved roughly 8.6 $\times$ best-valid speedup while retaining an advantage under a 10% write mix.

TPC-H Q19: write-aware multidimensional range aggregation. TPC-H Q19 computes total discounted revenue for three parametrized regions of the product space. Each branch constrains brand, container family, part size, and lineitem quantity, together with fixed shipping predicates. A normal execution scans lineitem,

joins it with `part`, applies the three predicate branches, and sums the surviving revenue even though the final result is a single scalar.

JetStream instead maintains a persistent join-derived revenue summary keyed by brand and container family, with each entry organized as a two-dimensional range structure over `part` size and `lineitem` quantity. The stored values represent summed discounted revenue from the corresponding joined `lineitem-part` region. At read time, each of the three SQL branches becomes one range-sum lookup, and the final answer is the sum of those three probes. This replaces the fact-table scan, join, and most of the predicate evaluation with a small number of logarithmic-time range queries.

The analysis stage also considered a two-dimensional prefix-sum representation, which would make reads even cheaper. It rejected that design because a point update would modify an unbounded suffix of the prefix structure, creating a large maintenance cost at higher scale factors. The selected range tree instead keeps both reads and point updates logarithmic, illustrating how JetStream trades a small amount of additional read work for substantially cheaper maintenance. The resulting accelerator achieved a best-valid dual-axis score of roughly 2638 $\times$ and remained hundreds of times faster under a 10% write mix.

4 Related Work

Synthesized accelerators. Bespoke OLAP [35] and GenDB [17] both propose techniques to generate specialized query execution code by leveraging LLMs. However, as discussed in Section 1, these systems have practical limitations. GenDB rewrites the underlying storage and generates query executors for pre-defined templates [17]. As a result, supporting writes requires re-running its storage optimization, and GenDB is unable to run unseen (e.g., ad-hoc) queries. Bespoke OLAP is similar, but it retains a fallback database system for unseen queries. This fallback effectively requires two copies of the data: one for accelerated queries, and one in the database system. Like GenDB, it also cannot support writes while preserving accelerated performance.

In contrast, JetStream integrates its accelerators into a full featured database system and relies on the database’s native storage. This design is practical: it requires only one copy of the data, queries for which an accelerator is available can use it directly, and unseen queries simply run unchanged on the underlying database. JetStream also leverages auxiliary state to accelerate specific queries, but it generates incremental maintenance logic for this state, allowing it to preserve most of its accelerated performance while delivering fresh results in the presence of writes.

GPT-DB [31] also generates code to execute a query, but does so interactively. Castor [11] is a classical approach to specializing the data layout to a static templated query workload; it does not support writes to the underlying data.

Database extensibility. Database extensions [10, 15, 26], user defined functions [5, 27], and declarative sub-operators [13] all provide mechanisms for adding functionality to a database system, which can include accelerators. However, with these mechanisms, end users are responsible for identifying and implementing the acceleration techniques, maintaining any auxiliary state, rewriting their queries to use the accelerators when beneficial, and porting their accelerator implementations to all the database systems they

want to support. Recent work on Tailwind [38] addresses most of these limitations by providing a framework for non-invasively integrating accelerators into database systems that support data import/export. Users define and implement parameterized accelerator families, and Tailwind then configures and selects accelerator instances for a given workload. JetStream goes further by using agents to automatically identify and implement acceleration opportunities and generate auxiliary-state maintenance logic. When possible, it uses UDFs and database extensions to integrate the resulting accelerators with the underlying database system.

Learned specialization. There is a long line of prior work that uses machine learning techniques to specialize a database system or data infrastructure to specific query workloads [8, 9, 16, 18, 19, 21, 25, 28, 32, 33, 37, 39, 40]. At a high level, these techniques define a parameterized space of possible system configurations and leverage learned models or analytical models to predict performance across this space, allowing the system parameters to be optimized for a given workload. JetStream is similar in spirit in that it specializes query execution to a given workload, but it does not require a manually defined parameterized configuration space; instead, LLM agents perform the workload analysis, code generation, and optimization.

Query compilation. Database systems such as Umbra [23], Hyper [14], Redshift [6], etc., also generate code to implement their chosen physical query plans [22]. However, classical query compilation is designed for generality: it uses a fixed code generation recipe for each operator in query plan [22]. In contrast, JetStream generates custom code that is specialized to a specific query and dataset, allowing it to employ query and data-specific tricks (e.g., bitpacking) and auxiliary state to accelerate the query.

Incremental view maintenance. JetStream incrementally maintains any auxiliary state it chooses to generate, which is similar to incremental view maintenance [3, 7, 12, 20, 41]. However, the key difference is that incremental view maintenance comprises algorithms for maintaining state computed from a SQL expression. In contrast, JetStream generates incremental maintenance code for arbitrary auxiliary state. In other words, the auxiliary state JetStream uses is not necessarily always a database view.

5 Conclusion

We presented JetStream, a framework for generating query-specific database accelerators that extend an existing DBMS, rather than replace it. JetStream combines a staged, measurement-driven synthesis workflow with a fixed engine-neutral substrate and generated backend adapters, allowing accelerators to execute directly over native database storage, and, when useful, maintain query-specific auxiliary state.

The resulting accelerators span a wide range of designs, from storage-aware stateless kernels to persistent indexes. Since JetStream treats reads and writes jointly, it can choose among incremental maintenance, rebuilds, and lazy repair while preserving the transactional semantics of the underlying database. Across established datasets and unseen workloads, JetStream is able to generate accelerators with large read speedups while retaining substantial gains under ongoing updates.

More broadly, our results suggest that LLM-based database specialization need not require synthesizing an entire replacement

engine or assuming static data. Instead, a production DBMS can remain responsible for canonical storage, transactions, recovery, and general SQL, while generated components only specialize the known expensive parts of a workload. This separation offers a practical path towards increasingly aggressive, workload-specific optimization without giving up the functionality and reliability of mature database systems.

References

- [1] Lior Abraham, John Allen, Oleksandr Barykin, Vinayak Borkar, Bhuwan Chopra, Ciprian Gerea, Daniel Merl, Josh Metzler, David Reiss, Subbu Subramanian, Janet L. Wiener, and Okay Zed. 2013. Scuba: Diving Into Data at Facebook. *Proceedings of the VLDB Endowment* 6, 11 (Aug. 2013), 1057–1067. doi:10.14778/2536222.2536231
- [2] Ankur Agiwal, Kevin Lai, Gokul Nath Babu Manoharan, Indrajit Roy, Jagan Sankaranarayanan, Hao Zhang, Tao Zou, Min Chen, Zongchang Chen, Ming Dai, Thanh Do, Haoyu Gao, Haoyan Geng, Raman Grover, Bo Huang, Yanlai Huang, Zhi Li, Jianyi Liang, Tao Lin, Li Liu, Yao Liu, Xi Mao, Yalan Meng, Prashant Mishra, Jay Patel, Rajesh S. R., Vijayshankar Raman, Sourashis Roy, Mayank Singh Shishodia, Tianhang Sun, Ye Tang, Junichi Tatemura, Sagar Trehan, Ramkumar Vadali, Prasanna Venkatasubramanian, Gensheng Zhang, Kefei Zhang, Yupu Zhang, Zeleng Zhuang, Goetz Graefe, Divyakant Agrawal, Jeff Naughton, Sujata Kosalge, and Hakan Hacigümüş. 2021. Napa: Powering Scalable Data Warehousing with Robust Query Performance at Google. *Proceedings of the VLDB Endowment* 14, 12 (July 2021), 2986–2997. doi:10.14778/3476311.3476377
- [3] Yanif Ahmad and Christoph Koch. 2009. DBToaster: A SQL Compiler for High-Performance Delta Processing in Main-Memory Databases. *Proceedings of the VLDB Endowment* 2, 2 (Aug. 2009), 1566–1569. doi:10.14778/1687553.1687592
- [4] Anthropic. 2025. Claude Code: Anthropic’s Agentic Coding System. <https://www.anthropic.com/product/claude-code>. Accessed: 2026-07-03.
- [5] Samuel Arch, Yuchen Liu, Todd C. Mowry, Jignesh M. Patel, and Andrew Pavlo. 2025. The Key to Effective UDF Optimization: Before Inlining, First Perform Outlining. *Proceedings of the VLDB Endowment* 18, 1 (Feb. 2025), 1–13. doi:10.14778/3696435.3696436
- [6] Nikos Armatatzoglou, Sanuj Basu, Naga Bhanoori, Mengchu Cai, Naresh Chainani, Kiran Chinta, Venkatraman Govindaraju, Todd J. Green, Monish Gupta, Sebastian Hellig, Eric Hotinger, Yan Leshinsky, Jintian Liang, Michael McCreedy, Fabian Nagel, Ippokratis Pandis, Panos Parchas, Rahul Pathak, Orestis Polychroniou, Foyzur Rahman, Gaurav Saxena, Gokul Soundararajan, Sriram Subramanian, and Doug Terry. 2022. Amazon Redshift Re-invented. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD ’22)*. 2205–2217. doi:10.1145/3514221.3526045
- [7] Mihai Budiu, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. 2023. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *Proceedings of the VLDB Endowment* 16, 7 (March 2023), 1601–1614. doi:10.14778/3587136.3587137
- [8] Jialin Ding, Ryan Marcus, Andreas Kipf, Vikram Nathan, Aniruddha Nrusimha, Kapil Vaidya, Alexander van Renen, and Tim Kraska. 2022. SageDB: An Instance-Optimized Data Analytics System. *Proceedings of the VLDB Endowment* 15, 13 (Sept. 2022), 4062–4078. doi:10.14778/3565838.3565857
- [9] Jialin Ding, Vikram Nathan, Mohammad Alizadeh, and Tim Kraska. 2020. Tsunami: A Learned Multi-Dimensional Index for Correlated Data and Skewed Workloads. *Proceedings of the VLDB Endowment* 14, 2 (2020), 74–86. doi:10.14778/3425879.3425880
- [10] DuckDB Authors. 2026. DuckDB Extensions. https://duckdb.org/docs/extensions/working_with_extensions.html.
- [11] John Feser, Sam Madden, Nan Tang, and Armando Solar-Lezama. 2020. Deductive Optimization of Relational Data Storage. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 170 (Nov. 2020), 30 pages. doi:10.1145/3428238
- [12] Jon Gjengset, Malte Schwarzkopf, Jonathan Behrens, Lara Timbó Araújo, Martin Ek, Eddie Kohler, M. Frans Kaashoek, and Robert Morris. 2018. Noria: dynamic, partially-stateful data-flow for high-performance web applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’18)*.
- [13] Michael Jungmair and Jana Giceva. 2023. Declarative Sub-Operators for Universal Data Processing. *Proceedings of the VLDB Endowment* 16, 11 (2023), 3461–3474. doi:10.14778/3611479.3611539
- [14] Alfons Kemper and Thomas Neumann. 2011. HyPer: A Hybrid OLTP&OLAP Main Memory Database System Based on Virtual Memory Snapshots. In *Proceedings of the 2011 IEEE 27th International Conference on Data Engineering (ICDE ’11)*. 195–206. doi:10.1109/ICDE.2011.5767867
- [15] Abigale Kim, Marco Slot, David G. Andersen, and Andrew Pavlo. 2025. Anarchy in the Database: A Survey and Evaluation of Database Management System Extensibility. *Proceedings of the VLDB Endowment* 18, 6 (Feb. 2025), 1962–1976. doi:10.14778/3725688.3725719
- [16] Ani Kristo, Kapil Vaidya, Ugur Çetintemel, Sanchit Misra, and Tim Kraska. 2020. The Case for a Learned Sorting Algorithm. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD ’20)*. 1001–1016. doi:10.1145/3318464.3389752
- [17] Jiale Lao and Immanuel Trummer. 2026. GenDB: The Next Generation of Query Processing – Synthesized, Not Engineered. arXiv:2603.02081 [cs.DB]
- [18] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the International Conference on Management of Data (SIGMOD ’21)*. doi:10.1145/3448016.3452838
- [19] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: A Learned Query Optimizer. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1705–1718. doi:10.14778/3342263.3342644
- [20] Derek G. Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martin Abadi. 2013. Naiad: A Timely Dataflow System. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP ’13)*. 439–455. doi:10.1145/2517349.2522738
- [21] Vikram Nathan, Jialin Ding, Mohammad Alizadeh, and Tim Kraska. 2020. Learning Multi-Dimensional Indexes. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD ’20)*. 985–1000. doi:10.1145/3318464.3380579
- [22] Thomas Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *Proceedings of the VLDB Endowment* 4, 9 (2011), 539–550. doi:10.14778/2002938.2002940
- [23] Thomas Neumann and Michael J. Freitag. 2020. Umbra: A Disk-Based System with In-Memory Performance. In *10th Conference on Innovative Data Systems Research (CIDR ’20)*. <https://vldb.org/cidrdb/2020/umbra-a-disk-based-system-with-in-memory-performance.html>
- [24] OpenAI. 2025. Introducing Codex. <https://openai.com/index/introducing-codex/>.
- [25] Andrew Pavlo, Matthew Butrovich, Lin Ma, Wan Shen Lim, Prashanth Menon, Dana Van Aken, and William Zhang. 2021. Make Your Database System Dream of Electric Sheep: Towards Self-Driving Operation. *Proceedings of the VLDB Endowment* 14, 12 (2021), 3211–3221. doi:10.14778/3476311.3476411
- [26] PostgreSQL Global Development Group. 2026. PostgreSQL Extensions. <https://www.postgresql.org/docs/current/external-extensions.html>.
- [27] Moritz Sichert and Thomas Neumann. 2022. User-Defined Operators: Efficiently Integrating Custom Algorithms into Modern Databases. *Proceedings of the VLDB Endowment* 15, 5 (Jan. 2022), 1119–1131. doi:10.14778/3510397.3510408
- [28] Sivaprasad Sudhir, Michael Cafarella, and Samuel Madden. 2021. Replicated Layout for In-Memory Database Systems. *Proceedings of the VLDB Endowment* 15, 4 (Dec. 2021), 984–997. doi:10.14778/3503585.3503606
- [29] Beth Barnes Sydney Von Arx, Lawrence Chan. 2025. Recent Frontier Models Are Reward Hacking. <https://metr.org/blog/2025-06-05-recent-reward-hacking/>.
- [30] Transaction Processing Performance Council. 2026. TPC-H Benchmark. https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-H_v3.0.1.pdf.
- [31] Immanuel Trummer. 2023. Demonstrating GPT-DB: Generating Query-Specific and Customizable Code for SQL Processing with GPT-4. *Proceedings of the VLDB Endowment* 16, 12 (Aug. 2023), 4098–4101. doi:10.14778/3611540.3611630
- [32] Kapil Vaidya, Subarna Chatterjee, Eric Knorr, Michael Mitzenmacher, Stratos Idreos, and Tim Kraska. 2022. SNARF: A Learning-Enhanced Range Filter. *Proceedings of the VLDB Endowment* 15, 8 (April 2022), 1632–1644. doi:10.14778/3529337.3529347
- [33] Kapil Vaidya, Eric Knorr, Michael Mitzenmacher, and Tim Kraska. 2021. Partitioned Learned Bloom Filters. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=6BRLOfrMhW>
- [34] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv preprint arXiv:2407.16741 (2024). <https://arxiv.org/abs/2407.16741>
- [35] Johannes Wehrstein, Timo Eckmann, Matthias Jasny, and Carsten Binnig. 2026. Bespoke OLAP: Synthesizing Workload-Specific One-size-fits-one Database Engines. arXiv:2603.02001 [cs.DB]
- [36] Jiacheng Yang, Ian Rae, Jun Xu, Jeff Shute, Zhan Yuan, Kelvin Lau, Qiang Zeng, Xi Zhao, Jun Ma, Ziyang Chen, Yuan Gao, Qilin Dong, Junxiong Zhou, Jeremy Wood, Goetz Graefe, Jeff Naughton, and John Cieslewicz. 2020. F1 Lightning: HTTP as a Service. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3313–3325. doi:10.14778/3415478.3415553
- [37] Zongheng Yang, Badrish Chandramouli, Chi Wang, Johannes Gehrke, Yinan Li, Umar Farooq Minhas, Per-Ake Larson, Donald Kossmann, and Rajeev Acharya. 2020. Qd-tree: Learning Data Layouts for Big Data Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD ’20)*. 193–208. doi:10.1145/3318464.3389770

- [38] Geoffrey X. Yu, Ryan Marcus, and Tim Kraska. 2026. Tailwind: A Practical Framework for Query Accelerators. arXiv:2604.28079 [cs.DB]
- [39] Geoffrey X. Yu, Markos Markakis, Andreas Kipf, Per-Åke Larson, Umar Farooq Minhas, and Tim Kraska. 2022. TreeLine: An Update-In-Place Key-Value Store for Modern Storage. *Proceedings of the VLDB Endowment* 16, 1 (2022), 99–112. doi:10.14778/3561261.3561270
- [40] Geoffrey X. Yu, Ziniu Wu, Ferdi Kossman, Tianyu Li, Markos Markakis, Amadou Ngom, Samuel Madden, and Tim Kraska. 2024. Blueprinting the Cloud: Unifying and Automatically Optimizing Cloud Data Infrastructures with BRAD. *Proceedings of the VLDB Endowment* 17, 11 (8 2024), 3629–3643. doi:10.14778/3681954.3682026
- [41] Jingren Zhou, Per-Åke Larson, and Hicham G Elmongui. 2007. Lazy Maintenance of Materialized Views. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB '07)*. 231–242. <https://www.vldb.org/conf/2007/papers/research/p231-zhou.pdf>